Task 6: Partition Strategy Benchmark for the Taxi Trips Dataset

Group 15: Hsi-Jui Chang, Xinyue Feng, Yixin Xia

September 12, 2026

1 Introduction

This report evaluates two Delta Lake storage designs for the `taxi_trips` dataset produced by the Task 3 ingestion framework, and compares them on ingestion time, storage size, file count, and the latency of the three required queries (trips per borough, average trip duration per day, average fare per borough). Partitioning determines how data is physically laid out on disk and therefore how much of it a query has to scan; a scheme that does not match the dataset's query patterns can be neutral at best and harmful at worst (e.g. by producing too many small files). This report tests that trade-off empirically.

2 Experimental Setup

2.1 Two storage strategies and environment

Both strategies write the exact same rows and schema; the only variable under test is the column(s) passed to `.partitionBy(...)`.

- **Strategy A (time-based):** partitioned by `pickup_year` and `pickup_month`, matching the scheme already used for the dataset's primary copy from Task 3.
- **Strategy B (location-based):** partitioned by `pickup_borough`, derived by joining the trips table against the `taxi_zones` lookup table on `pickup_location_id`.

Both strategies call `.repartition(...)` on the same column(s) before writing, so rows sharing a partition value land in one shuffle output instead of being scattered across Spark's shuffle partitions.

All experiments ran on a single machine using Spark 4.2.0 with Delta Lake 4.4.0, in local mode (`local[*]`), with a 12 GB driver heap and `spark.sql.shuffle.partitions` set to 64. The source dataset contains 9,417,689 trip records.

3 Evaluation

3.1 Data distribution and quality

Before interpreting any benchmark numbers, Tables 1 and 2 show how the source data is actually distributed across each partitioning key.

January–March 2024 account for 9,417,669 of 9,417,689 rows (99.9998%). The other five groups — 2002, 2008, 2009, December 2023, and even April 2024 — hold only 20 rows combined, almost certainly malformed timestamps rather than genuine trips (a 2002 NYC taxi trip predates the dataset's stated 2024 scope by 22 years). They were left in place rather than filtered out, since removing them was outside the scope of Task 6, but are worth flagging for the ingestion framework (Task 3) to validate against.

Manhattan alone accounts for 89.65% of all trips, which is the single fact that explains most of the benchmark results below.

Year	Month	Rows
2024	3	3,523,884
2024	2	2,966,792
2024	1	2,926,993
2023	12	10
2009	1	4
2002	12	3
2024	4	2
2008	12	1

Table 1: Every (`pickup_year`, `pickup_month`) group in `taxi_trips`.

Borough	Rows	% of total
Manhattan	8,442,887	89.65%
Queens	817,093	8.68%
Brooklyn	95,770	1.02%
Unknown	31,304	0.33%
Bronx	24,727	0.26%
N/A	4,773	0.05%
EWR	915	0.01%
Staten Island	220	0.002%

Table 2: Trips by `pickup_borough`. `N/A` (zone "Outside of NYC") and `Unknown` (no matching zone at all) are kept as separate categories rather than merged, since the former is a deliberate, informative label and the latter is a genuine gap.

3.2 Storage and ingestion time

Table 3 reports the results of writing the full dataset once under each strategy. Delta writes one subdirectory per distinct partition value (e.g. `pickup_borough=Manhattan/`); values containing characters unsafe in file paths are URL-encoded, so the `N/A` borough is written as `pickup_borough=N%2FA/`.

Strategy	Ingestion time (s)	Storage size (MB)	File count
A — time-based (year, month)	12.24	842.8	16
B — location-based (borough)	22.88	845.8	16

Table 3: Write-time metrics for each partitioning strategy.

Storage size and file count are nearly identical, which was not the pattern we expected: with only 8 boroughs versus dozens of year/month combinations, we expected Strategy B to produce noticeably fewer, larger files. Both produced exactly 16. This is best explained by Adaptive Query Execution's partition-coalescing (`spark.sql.adaptive.coalescePartitions`, default-enabled in Spark 4.x), which merges shuffle output up to an advisory target size (≈64 MB) independently of the partitioning column; ≈843 MB of output coalescing to ≈16 files is consistent with that target regardless of key cardinality.

Ingestion time, by contrast, differs by 87% (22.88s vs. 12.24s). Since Spark's repartition step assigns all rows sharing a key to one task, and the slowest task sets the whole stage's time, the relevant number is each strategy's single *largest* group: Manhattan (8,442,887 rows, Table 2) is about 2.4× the size of the largest month group, March 2024 (3,523,884 rows, Table 1) — a ratio that lines up with the ≈1.9× gap in measured write time.

3.3 Query latency

Table 4 reports the latency of the three required queries, each timed over 5 repetitions with run order alternated each time and the median reported, after warming up both strategies with a discarded pass — this avoids one strategy benefiting from JVM warm-up merely by running first.

Query	Strategy	Median latency (s)
Trips per borough	A (time)	0.139
	B (borough)	0.119
Avg. trip duration per day	A (time)	0.142
	B (borough)	0.147
Avg. fare per borough	A (time)	0.148
	B (borough)	0.133

Table 4: Query latency under each partitioning strategy (median of 5 runs, alternating which strategy runs first).

Strategy B is faster on both borough-grouped queries, and Strategy A is faster on the day-grouped query, the expected direction, since a scheme helps most when a query's grouping column matches it. Differences are small (0.005–0.02 s): none of the three queries filter on the partition column, so pruning (which works by skipping directories a filter rules out) never triggers — a full scan touches every partition regardless of scheme.

3.4 Partition pruning

To verify pruning rather than assert it, we inspected Spark's physical plan: filtering on the partition column produces a non-empty `PartitionFilters` list, while filtering on any other column (e.g. `passenger_count`) produces an empty one instead, with the same condition appearing as a `DataFilter` instead. This confirms pruning triggers specifically for the partition column, exactly as the mechanism predicts.

We also attempted to measure pruning's wall-clock benefit by timing a full scan against a filter on each strategy's majority and minority partition value. This did not produce a usable result: once a strategy's files are read once in a session, Spark and the OS file-system cache keep them warm for the rest of the notebook run, so any repeated-measurement design that controls for JVM warm-up and run order (as used successfully in Table 4) ends up warming the full scan itself before the timed phase begins, erasing the very I/O difference the test exists to detect. At this dataset's size (≈843 MB total, comfortably cached by a 48 GB machine), a full scan and a highly selective filter become indistinguishable in wall-clock time within the same session, and a Jupyter kernel restart does not clear the OS-level page cache that causes this. This is a limitation of measuring I/O savings on a small dataset with abundant RAM, not a failure of the pruning mechanism itself, which the physical-plan inspection above already confirms works correctly.

4 Conclusions

Three points stand out. First, storage size and file count were governed by Adaptive Query Execution's partition coalescing rather than the number of distinct partition values, giving both strategies near-identical file counts despite very different key cardinalities. Second, the ingestion-time gap traces to each strategy's single largest group (Manhattan vs. March 2024), not overall skew. Third, pruning is confirmed to trigger correctly on the partition column via Spark's physical plan, but its wall-clock benefit could not be reliably measured at this dataset's scale: once files are read once in a session, Spark and OS caching keep them warm for the rest of the notebook run, erasing the I/O difference between a full scan and a highly selective filter. The three required queries are unfiltered full-table aggregations regardless, so pruning would not have benefited them even if it were measurable.